

File Intake Automation

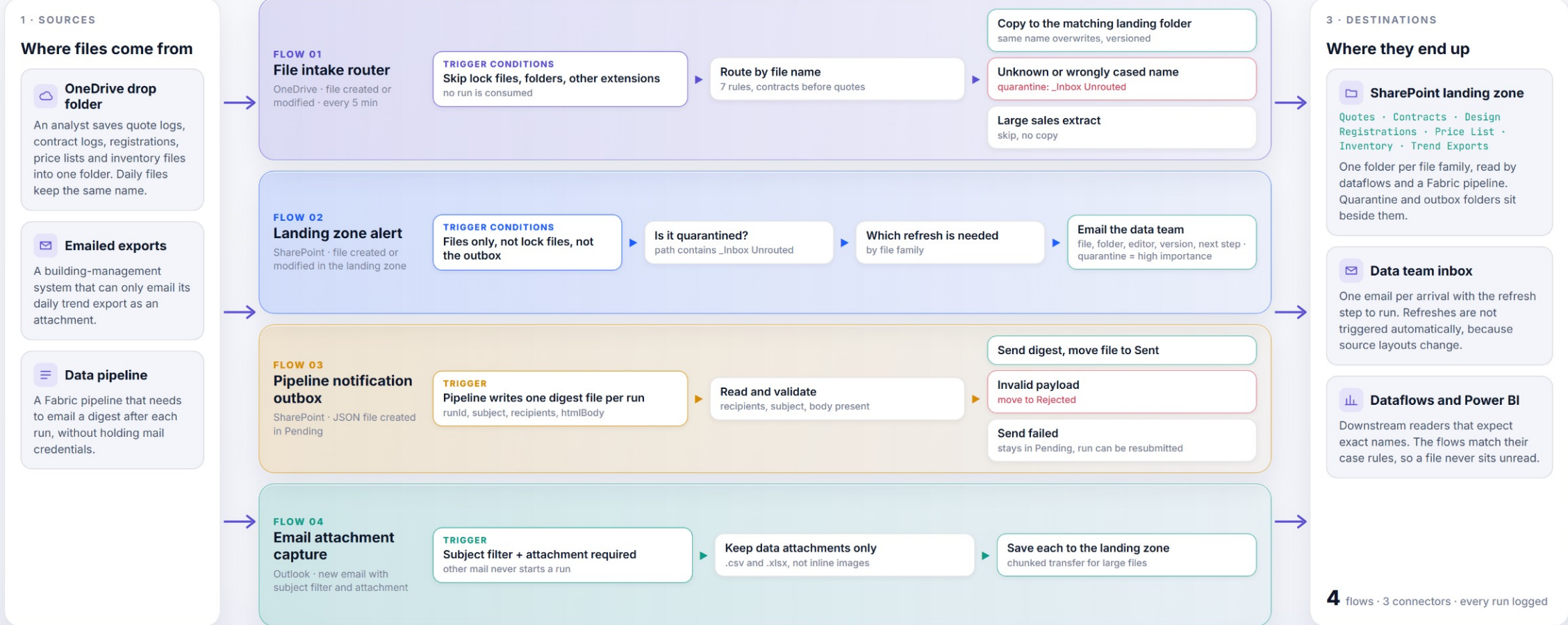
Routing · Quarantine · Alerts · Pipeline hand-off

POWER AUTOMATE

File Intake and Notification Flows

Business files arrive by folder drop and by email; four cloud flows route them, alert the data team and send what a pipeline asks them to send

— Flow steps Standard connectors only · no Premium licence



FILTER AT THE TRIGGER
Noise never starts a run
Lock files, folders, wrong extensions and unrelated emails are rejected by trigger conditions, so run quota and run history stay clean.

MATCH THE READER
Case policy per file family
Most rules are case-insensitive. Inventory names are case-exact because the dataflow that reads them is. Rule order handles names that contain other names.

FAIL VISIBLY
Quarantine plus an alert
A file that matches no rule is copied to _Inbox Unrouted and emailed at high importance, instead of landing where nothing reads it.

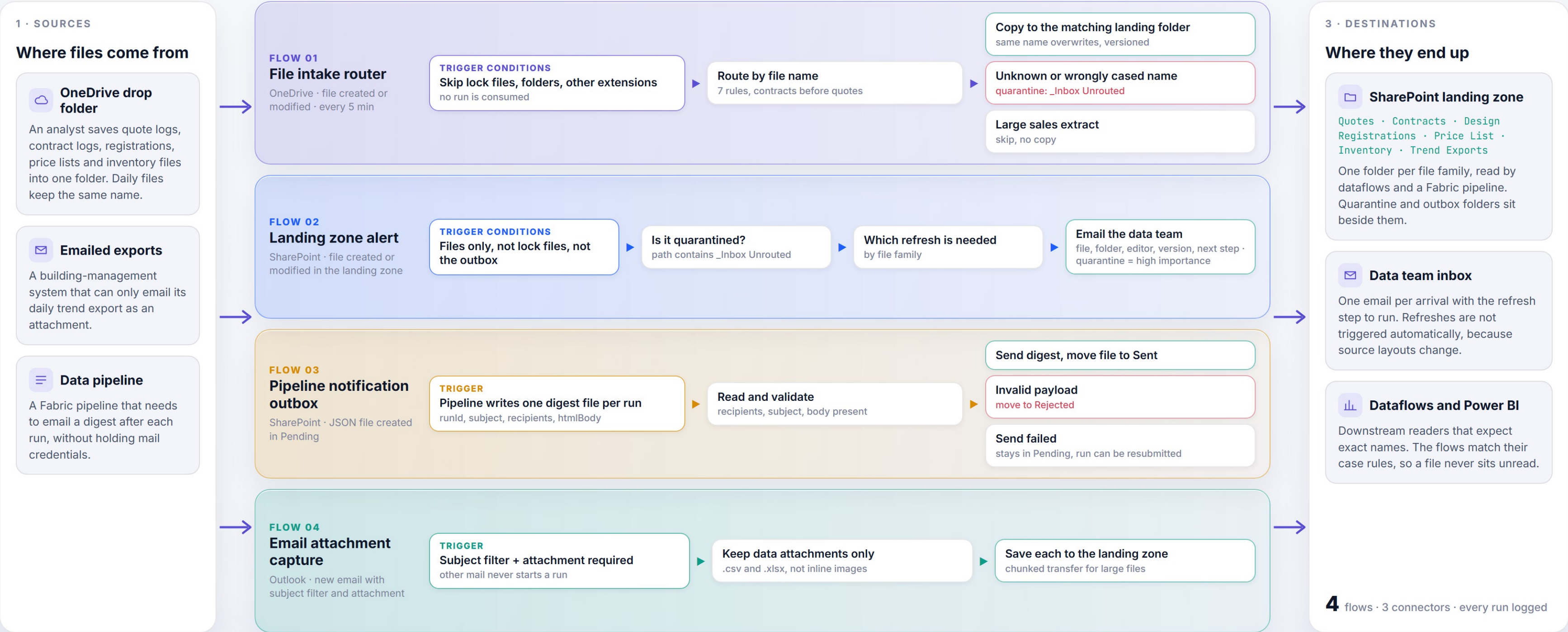
HAND-OFF CONTRACT
The folder is the status
The pipeline holds no mail credentials and never calls a Premium endpoint. Pending → Sent / Rejected tells it what happened.

File Intake and Notification Flows

Business files arrive by folder drop and by email; four cloud flows route them, alert the data team and send what a pipeline asks them to send

— Flow steps

----- Standard connectors only · no Premium licence



FILTER AT THE TRIGGER

Noise never starts a run

Lock files, folders, wrong extensions and unrelated emails are rejected by trigger conditions, so run quota and run history stay clean.

MATCH THE READER

Case policy per file family

Most rules are case-insensitive. Inventory names are case-exact because the dataflow that reads them is. Rule order handles names that contain other names.

FAIL VISIBLY

Quarantine plus an alert

A file that matches no rule is copied to `_Inbox Unrouted` and emailed at high importance, instead of landing where nothing reads it.

HAND-OFF CONTRACT

The folder is the status

The pipeline holds no mail credentials and never calls a Premium endpoint. `Pending` → `Sent` / `Rejected` tells it what happened.

Files route themselves, and mistakes are visible

An analyst drops files into one OneDrive folder. Each file is copied to the landing folder its name says it belongs in, so the dataflows that read those folders never need a person in between.

01 Noise is rejected before a run starts

Trigger conditions drop Office lock files, folders and unsupported extensions. A stray `.txt` file consumed no run at all.

02 Rule order and case are deliberate

Contract logs are named *Quote Log Customer Contracts*, so the contract rule is tested before the quote rule. Inventory names are case-exact because the dataflow that reads them is.

03 Unknown names are quarantined, not lost

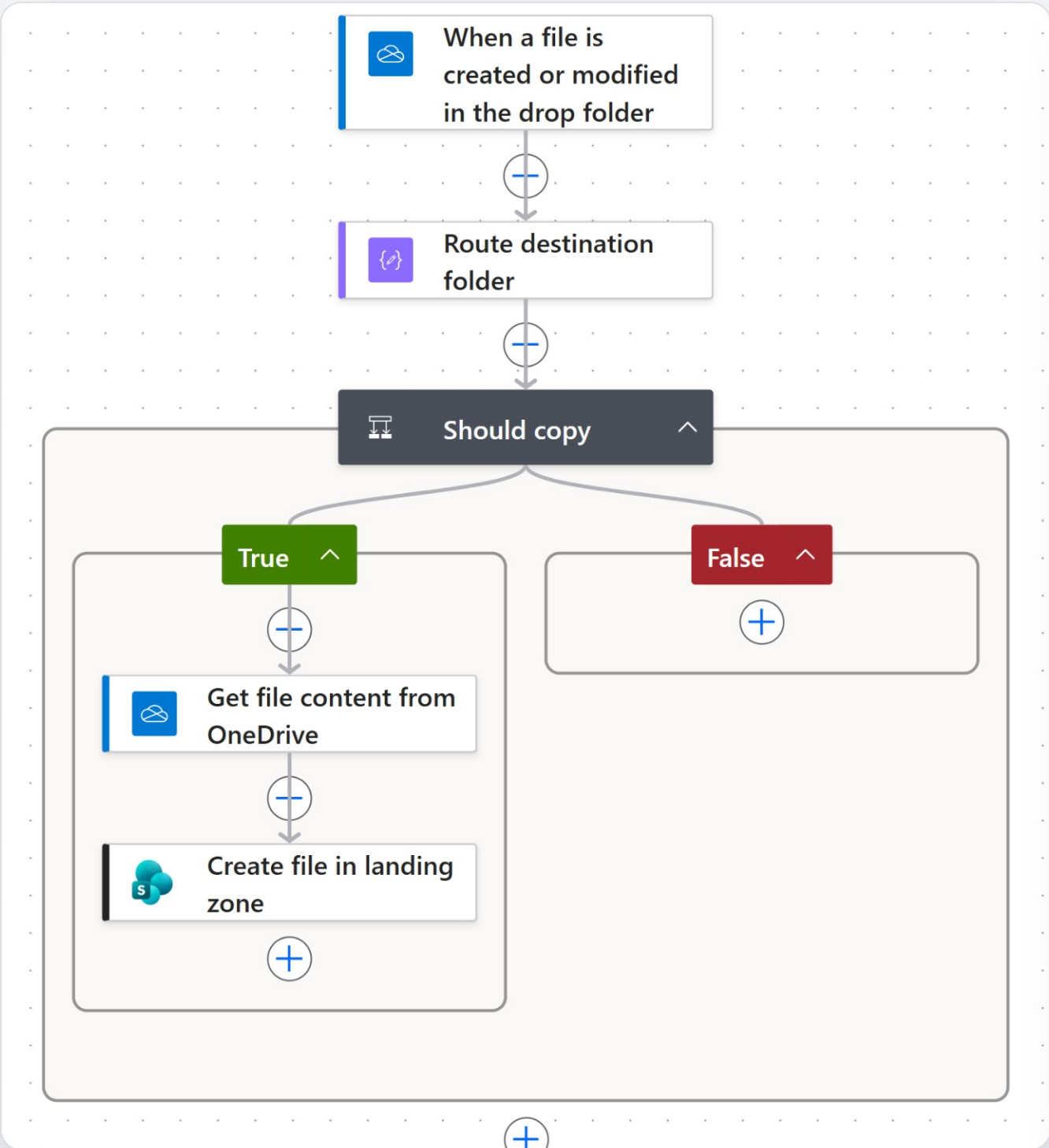
A file that matches no rule goes to `_Inbox Unrouted` and triggers a high-importance alert, instead of landing where nothing reads it.

18 of 18

live runs succeeded, including overwrites

16 / 16

routing test cases pass



Left: flow canvas. Trigger conditions, a routing expression, and a copy guarded by a condition. Right: a real run. The routing step's output for a wrongly cased inventory file is the quarantine folder.

Route destination folder

Order matters: contract logs are tested before 'quote log' because their names contain it. Inventory names are case-exact to match the downstream reader; wrong case is quarantined. Large sales extracts are skipped.

Run results | Settings | Code view | About

INPUTS

Inputs

/Documents/Portfolio Demo/Landing/_Inbox Unrouted

OUTPUTS

Outputs

/Documents/Portfolio Demo/Landing/_Inbox Unrouted

PROPERTIES

Start time

10/6/2026, 1:58:18 AM (Local time)

End time

10/6/2026, 1:58:18 AM (Local time)

Status

Succeeded

Client tracking ID

Tested with real files, checked by hand

Seven files with production-style names were dropped three times. Every run is in the flow's history, and every file is where the rules say it should be.

01 Routing rules in one place

Seven ordered rules in a single expression, mirrored by a Python test so a rule change is checked before deployment.

02 Same-name uploads overwrite

Daily files keep their name. The destination is replaced and SharePoint keeps the previous copy as a version.

03 Every run is accounted for

Six files, six runs per drop; the filtered file produced none. All eighteen runs succeeded in under a second each.

#	NAME CONTAINS	CASE	DESTINATION	
1	customer contract log	any	Contracts	copy
2	quote log	any	Quotes	copy
3	design registration...	any	Design Registrations	copy
4	price list	any	Price List	copy
5	Distributor Backlog · Month-End Inventory	exact	Inventory	special
6	ytd sales detail	any	skip (manual monthly upload)	special
7	anything else		_Inbox Unrouted + alert	quarantine

Routing contract. Order matters: rule 1 is tested before rule 2 because contract-log names contain “quote log”.

28-day run history ⓘ

Edit columnsAll runs

Start	Duration	Status
Oct 6, 01:58 AM (10 h ago)	57 ms	Succeeded
Oct 6, 01:58 AM (10 h ago)	694 ms	Succeeded
Oct 6, 01:58 AM (10 h ago)	711 ms	Succeeded
Oct 6, 01:58 AM (10 h ago)	740 ms	Succeeded
Oct 6, 01:58 AM (10 h ago)	682 ms	Succeeded
Oct 6, 01:58 AM (10 h ago)	628 ms	Succeeded
Oct 6, 01:28 AM (10 h ago)	94 ms	Succeeded
Oct 6, 01:28 AM (10 h ago)	722 ms	Succeeded

My files > Portfolio Demo > Landing ▾

ShareCopy linkDownloadAutomate ▾

Name ▾	Modified ▾	Modified By ▾	File size ▾	Sharing ▾	Activity
Trend Exports	Yesterday	Salman Hussain	2 items	Private	
_Notifications	Yesterday	Salman Hussain	3 items	Private	
_Contracts	Yesterday	Salman Hussain	1 item	Private	
_Quotes	Yesterday	Salman Hussain	1 item	Private	
Design Registrations	Yesterday	Salman Hussain	1 item	Private	
_Inbox Unrouted	Yesterday	Salman Hussain	2 items	Private	

Left: run history, all succeeded. Right: the landing zone after the test, one folder per file family plus quarantine and outbox.

7

routing rules

< 1s

typical run

The team learns what arrived and what to run

Every file that reaches the landing zone produces one email: what it is, who put it there, which version, and the refresh it needs. Refreshes are deliberately not triggered automatically.

01 One flow, two tones

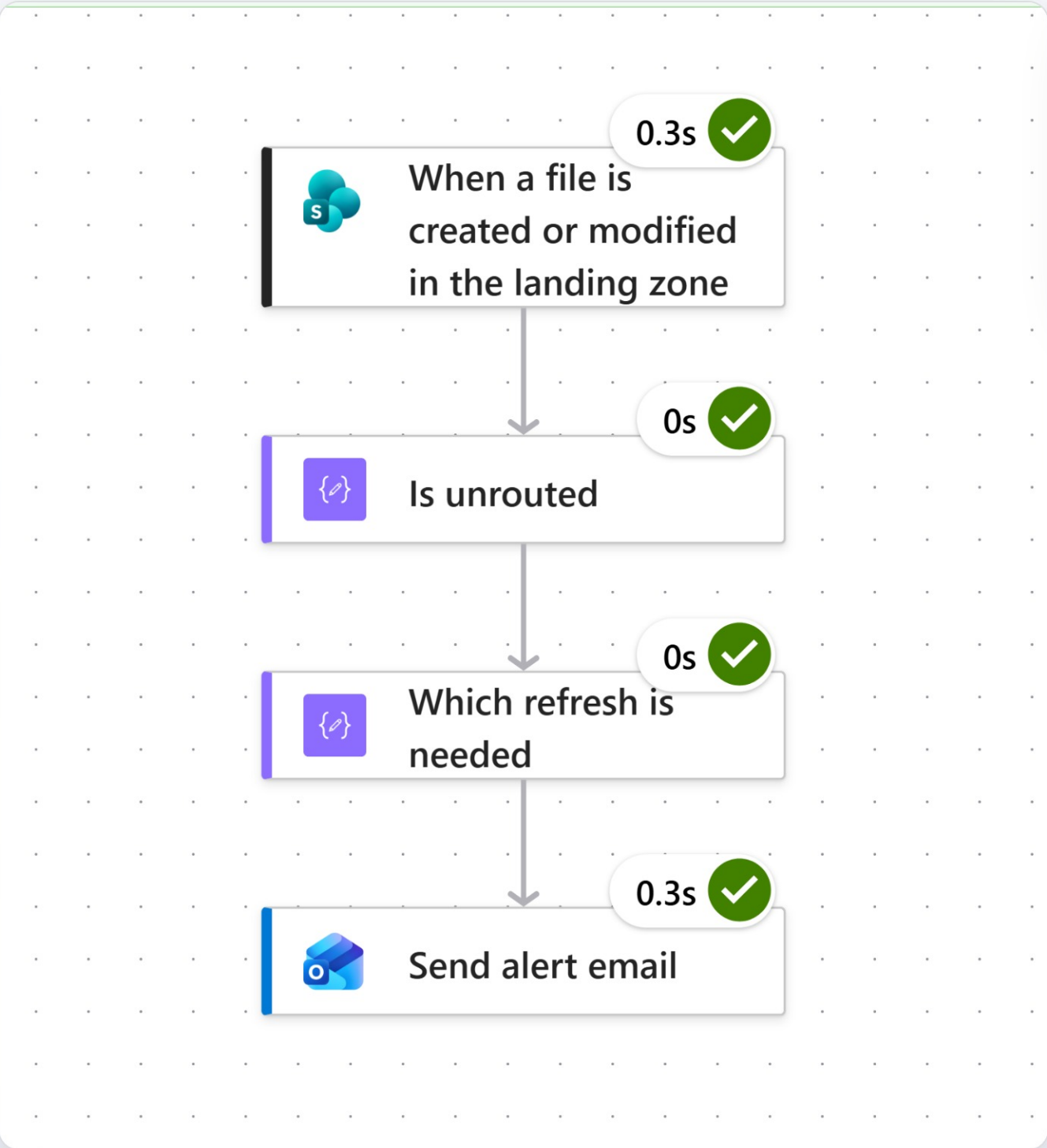
Normal uploads arrive as ordinary mail. Quarantined files arrive marked UNROUTED at high importance, so they are noticed.

02 The next step is in the email

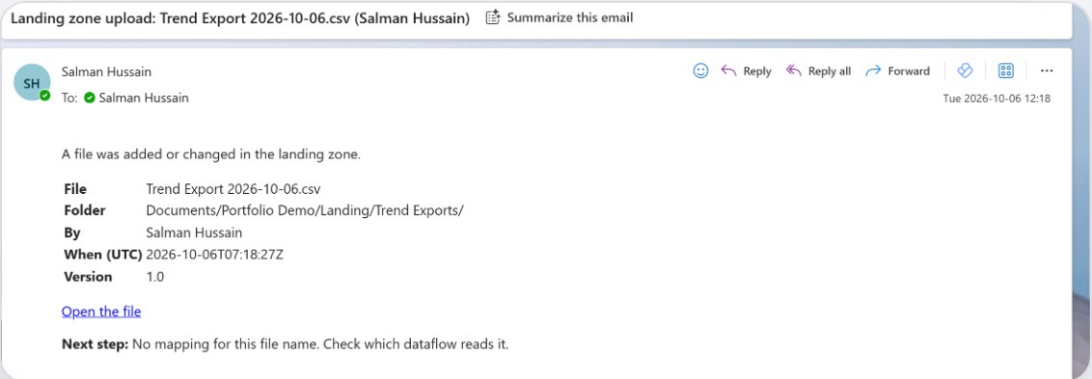
A small expression maps the file family to the dataflow or pipeline that reads it, so whoever is on duty does not have to look it up.

03 Why not refresh automatically

Source file layouts change without warning. A person looks at the file first; the alert makes that quick.



Run view. Trigger, two expressions, one email, all green.



The email. File, folder, editor, time, version, a link, and the refresh step to run.

FILE FAMILY	NEXT STEP IN THE EMAIL
Quote log, contract log, price list, registrations	Run the quotes ingestion pipeline, then refresh the Quotes and Sales models
Distributor backlog, month-end inventory	Run the inventory dataflow, then refresh the Sales model
YTD sales detail	Run the sales dataflow, then refresh the Sales model
Quarantined	Check the name (inventory names are case-sensitive), rename, drop again
Anything else	No mapping: check which dataflow reads it

Refresh map. One expression in the flow; the email carries the answer.

7 of 7

runs, one email each

5 min

polling interval

A data pipeline sends email without owning a mailbox

A Fabric pipeline renders a digest and writes it as a JSON file. The flow sends it and moves the file, so the pipeline can see what happened without an HTTP endpoint or a Premium licence.

01 No credentials in the data platform

The flow owns the mail connection. The pipeline only writes a file: `runId`, `subject`, `recipients`, `htmlBody`.

02 The folder is the status

`Sent` means delivered, `Rejected` means the payload was incomplete, and a file still in `Pending` means the send failed and the run can be resubmitted.

03 One receiving flow instead of twelve

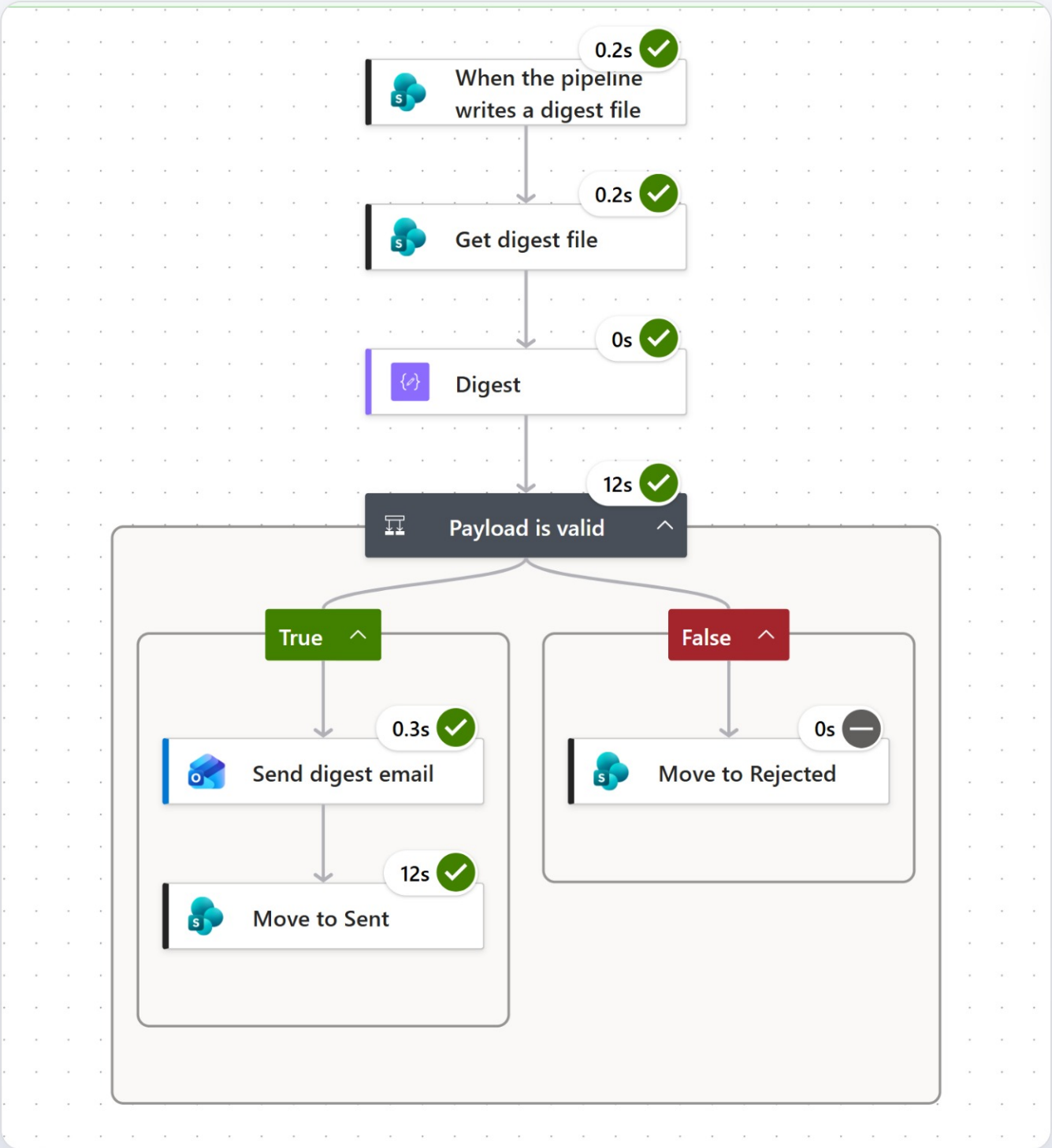
In the original engagement this replaced a per-region set of polling flows with one flow that does nothing but send what it is given.

2 of 2

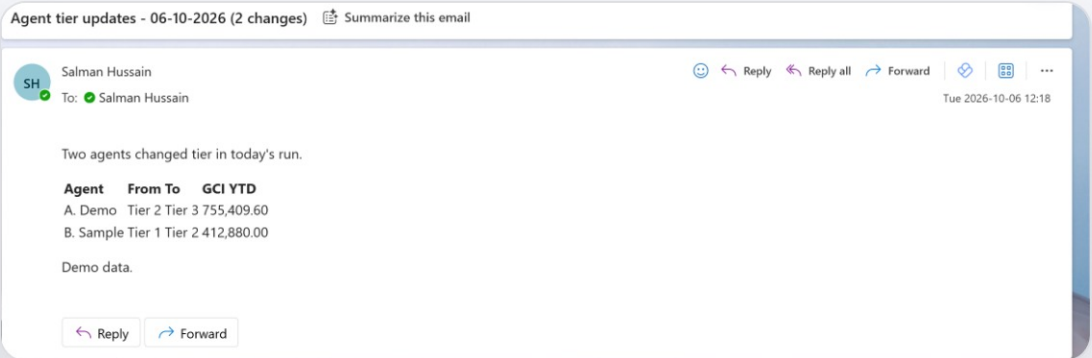
paths tested: Sent and Rejected

0

Premium connectors



Run view. A valid digest: read, validated, sent, moved to Sent. The Rejected branch was skipped.



The digest. Rendered by the pipeline, delivered by the flow.

```
{
  "runId": "20261006T030014Z-9ff268b8",
  "subject": "Agent tier updates - 06-10-2026 (2 changes)",
  "recipients": ["reporting@example.com", "ops@example.com"],
  "htmlBody": "<p>Two agents changed tier in today's run.</p>..."
}
```

The contract. One JSON file per run in `Pending`. Missing fields send it to `Rejected`; a failed send leaves it in `Pending` for a resubmit.

Systems that can only email still feed the platform

A building-management system sends its daily trend export as an attachment. The flow saves it to the landing zone, where the same alert and dataflows pick it up.

01 Filtered at the trigger

A subject filter and an attachment requirement sit on the trigger itself, so the rest of the mailbox never starts a run.

02 Only data attachments are saved

Inline images and signatures are filtered out; only .csv and .xlsx attachments reach the folder.

03 Large files are handled

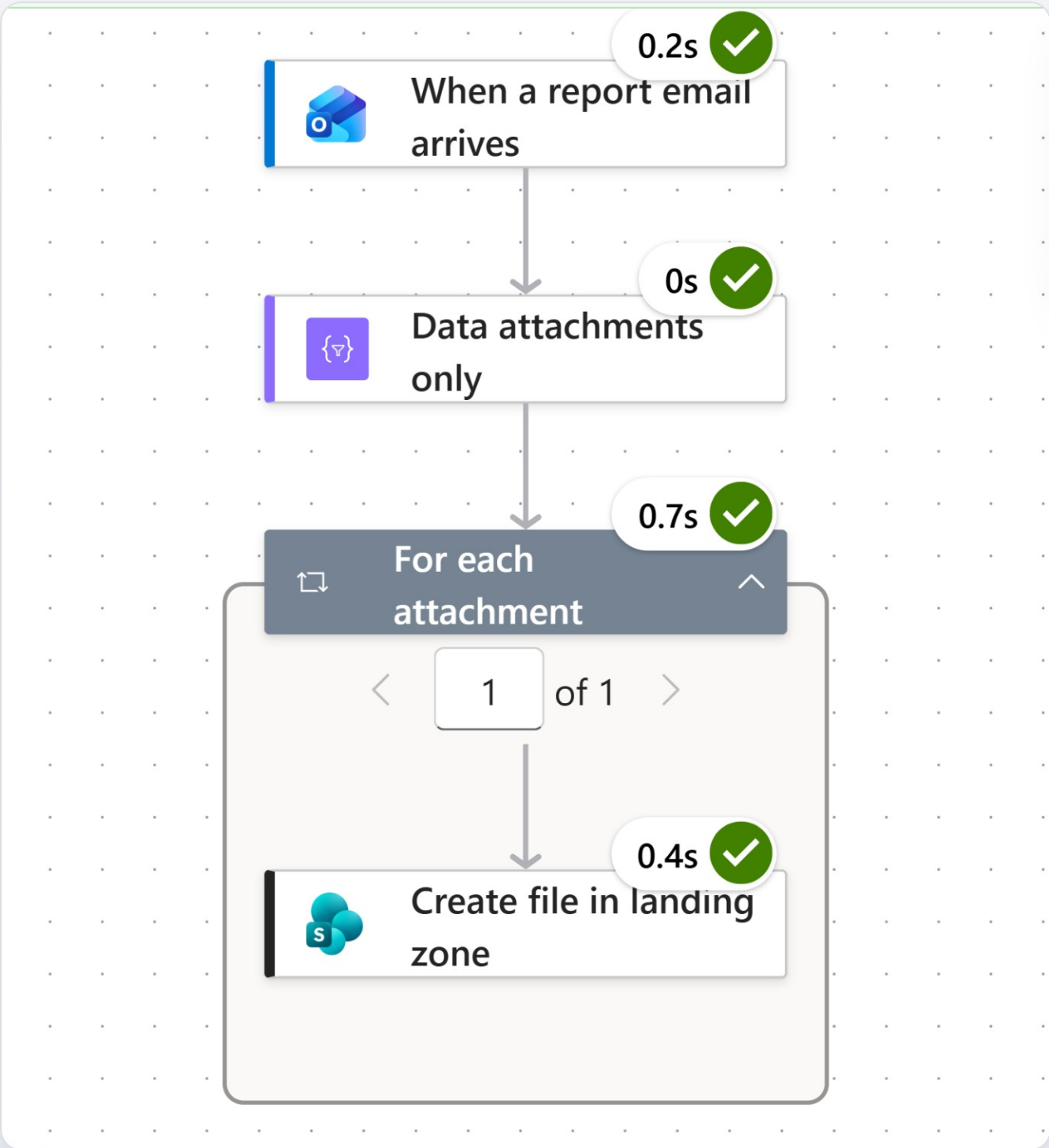
The copy uses chunked transfer, so multi-megabyte exports do not fail on size.

1 of 2

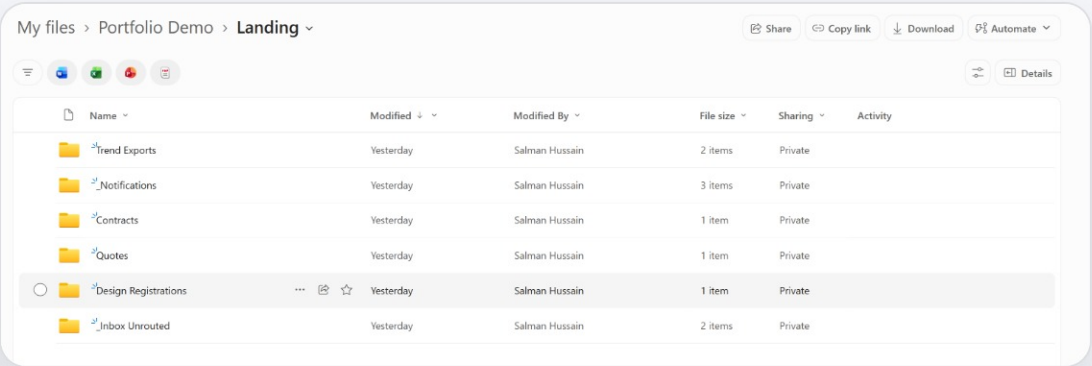
attachments saved: the CSV, not the image

< 2 s

from email to folder



Run view. One loop iteration: the image attachment was filtered out before the loop.



The landing zone. Trend Exports sits beside the folders fed by the drop-folder flow, so the same alert and dataflows apply.

ATTACHMENT IN THE TEST EMAIL	FILTER	OUTCOME
Trend Export 2026-10-06.csv	.csv, not inline	Saved to Trend Exports; alert email sent by flow 02
logo.png	not a data file	Dropped by the filter array; nothing written

Filter array. `not(isInline)` and `(endsWith .csv or .xlsx)`, evaluated once per email.

What you get with this build

The same habits apply whether the files come from an analyst, a vendor portal, a building system or a pipeline: filter early, route by rule, fail visibly, hand off cleanly.

✓

Filter at the trigger
Lock files, folders, wrong extensions and unrelated mail never consume a run.

✓

Rules match the reader
Case policy and rule order follow the dataflows that read each folder.

✓

Fail visibly
Unknown files are quarantined and alerted, never dropped silently.

✓

Safe to re-run
Same-name copies overwrite with versioning; a failed send stays Pending.

✓

No credentials downstream
Pipelines hand off to a flow instead of holding mail secrets.

✓

Standard connectors
OneDrive, SharePoint and Outlook only. No Premium licence needed.

✓

Tested before go-live
Routing simulated in code, then exercised live with real file names.

✓

Definitions as code
Each flow is a JSON definition, deployed by script with placeholders for your tenant.

FLOW	TRIGGER	LIVE TEST	RESULT
01 File intake router	OneDrive file created or modified	7 files dropped three times	18 runs, every file in the right folder, overwrites clean
02 Landing zone alert	SharePoint file created or modified	Every landing-zone arrival	7 runs, 7 emails, quarantine flagged at high importance
03 Pipeline notification outbox	JSON file created in Pending	Valid and invalid digests	Sent and Rejected paths both exercised
04 Email attachment capture	Email with subject filter and attachment	Email with a CSV and an image	Only the CSV saved, chunked transfer

4

cloud flows

3

standard connectors

32

live runs, 0 failed

7

routing rules, 16 test cases

0

Premium licences required